check_charts

October 22, 2026

[1]:
```
import pandas as pd
import glob
import os

# Find all sweep CSV files in the data directory
csv_files = sorted(glob.glob("../data/sweep_*.csv"))

if not csv_files:
    print("No sweep CSV files found in the 'data/' directory.")
else:
    dataframes = {}
    for file in csv_files:
        filename = os.path.basename(file)
        # comment='#' automatically skips the metadata lines
        df = pd.read_csv(file, comment='#')
        dataframes[filename] = df

        # Print a quick summary of what was loaded
        freqs = df['freq_hz'].unique()
        print(f"  Loaded: {filename}")
        print(f"    Rows: {len(df)} | Frequencies: {len(freqs)} ({freqs[0]:.0f}␣
 ↪to {freqs[-1]:.0f} Hz)\n")

# To access a specific file later, use:
# df = dataframes['sweep_xiao_20261016_144657_50step.csv']
```

```
  Loaded: sweep_xiao_20261016_122345_10step.csv
    Rows: 301205 | Frequencies: 100 (10 to 1000 Hz)

  Loaded: sweep_xiao_20261016_144327_5step.csv
    Rows: 575279 | Frequencies: 199 (10 to 1000 Hz)

  Loaded: sweep_xiao_20261016_144548_20step.csv
    Rows: 166245 | Frequencies: 50 (10 to 990 Hz)

  Loaded: sweep_xiao_20261016_144657_50step.csv
    Rows: 89230 | Frequencies: 20 (10 to 960 Hz)
```

```
Loaded: sweep_xioa_20261016_123137_1step.csv
  Rows: 2778878 | Frequencies: 991 (10 to 1000 Hz)
```

[2]:
```python
import matplotlib.pyplot as plt
import numpy as np

def plot_reconstructed_signals(df, max_plots=9):
    """
    Plots measured vs ideal reconstructed signal for each frequency segment.
    """
    frequencies = sorted(df['freq_hz'].unique())
    n_freqs = min(len(frequencies), max_plots)

    cols = 3
    rows = (n_freqs + cols - 1) // cols

    fig, axes = plt.subplots(rows, cols, figsize=(15, 5 * rows))

    # Flatten axes array for easy iteration, handle edge case of 1 plot
    if n_freqs == 1:
        axes = [axes]
    else:
        axes = axes.flatten()

    for i, freq in enumerate(frequencies[:max_plots]):
        ax = axes[i]
        segment = df[df['freq_hz'] == freq]

        # Normalize time to start at 0 ms for this specific segment for cleaner␣
↪viewing
        t_rel = segment['time_ms'] - segment['time_ms'].iloc[0]

        # Plot Ideal vs Measured
        ax.plot(t_rel, segment['gen_ref_v'], label='Ideal (gen_ref_v)',␣
↪color='gray', linestyle='--', linewidth=1.5)
        ax.plot(t_rel, segment['adc_v'], label='Measured (adc_v)', color='tab:
↪blue', marker='.', markersize=2, linewidth=1)

        ax.set_title(f"Reconstruction: {freq:.1f} Hz")
        ax.set_xlabel("Time (ms)")
        ax.set_ylabel("Voltage (V)")
        ax.legend(loc='upper right', fontsize=8)
        ax.grid(True, alpha=0.3)

    # Hide any unused subplots
    for j in range(i + 1, len(axes)):
```

```
        axes[j].axis('off')

    plt.suptitle("Signal Reconstruction: Measured ADC vs Ideal Generator␣
 ↪Reference", fontsize=14, fontweight='bold')
    plt.tight_layout(rect=[0, 0, 1, 0.96])
    plt.show()

# Example usage (uncomment the file you want to analyze):
# plot_reconstructed_signals(dataframes['sweep_xiao_20261016_144657_50step.
 ↪csv'], max_plots=9)
```

[3]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import rfft, rfftfreq

def analyze_fft_and_spectrogram(df, target_freq_for_detail=510.0):
    """
    Analyzes FFT quality across the sweep and generates a 2D Spectrogram
    to show all frequencies in a single, comprehensive plot.
    """
    frequencies = sorted(df['freq_hz'].unique())

    # --- PART 1: Summary Table ---
    print(f"{'Target Freq (Hz)':<15} | {'Detected Peak (Hz)':<18} | {'Magnitude␣
 ↪(V)':<15} | {'Error (Hz)':<10}")
    print("-" * 65)

    # Arrays to store data for the Spectrogram
    spectrogram_magnitudes_interp = []
    f_common = np.arange(0, 1500, 1) # Common axis: 0 to 1500 Hz, 1 Hz␣
 ↪resolution

    for freq in frequencies:
        segment = df[df['freq_hz'] == freq].copy()

        # Calculate actual sampling rate from time steps
        dt_ms = segment['time_ms'].diff().dropna().mean()
        fs = 1000.0 / dt_ms
        N = len(segment)

        # Remove DC offset for cleaner FFT
        adc_signal = segment['adc_v'].values - np.mean(segment['adc_v'].values)

        # Compute FFT
        yf = rfft(adc_signal)
        xf = rfftfreq(N, d=(1.0/fs))
        magnitude = np.abs(yf) / N
```

```
        # INTERPOLATE onto the common frequency axis so all arrays have the␣
↪same length
        mag_interp = np.interp(f_common, xf, magnitude)
        spectrogram_magnitudes_interp.append(mag_interp)

        # Find peak (skip index 0 which is DC)
        peak_idx = np.argmax(magnitude[1:]) + 1
        peak_freq = xf[peak_idx]
        peak_mag = magnitude[peak_idx]
        error = abs(peak_freq - freq)

        print(f"{freq:<15.1f} | {peak_freq:<18.2f} | {peak_mag:<15.4f} | {error:
↪<10.2f}")

    # --- PART 2: Single Clean FFT Plot (for spectral purity proof) ---
    detail_segment = df[df['freq_hz'] == target_freq_for_detail].copy()
    dt_ms = detail_segment['time_ms'].diff().dropna().mean()
    fs = 1000.0 / dt_ms
    N = len(detail_segment)

    yf_detail = rfft(detail_segment['adc_v'].values - np.
↪mean(detail_segment['adc_v'].values))
    xf_detail = rfftfreq(N, d=(1.0/fs))
    mag_detail = np.abs(yf_detail) / N

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    # Plot 1: Clean FFT of a single frequency
    ax1.plot(xf_detail, mag_detail, color='tab:blue', linewidth=1.5)
    ax1.axvline(target_freq_for_detail, color='red', linestyle='--',␣
↪label=f'Target: {target_freq_for_detail} Hz')
    ax1.set_title(f"FFT Spectral Purity at {target_freq_for_detail} Hz")
    ax1.set_xlabel("Frequency (Hz)")
    ax1.set_ylabel("Magnitude (V)")
    ax1.set_xlim(0, fs / 2)
    ax1.grid(True, alpha=0.3)
    ax1.legend()

    # --- PART 3: 2D Spectrogram Heatmap (All frequencies in one plot) ---
    # Now that all arrays are interpolated to the same length, we can stack␣
↪them!
    spectrogram_magnitudes = np.array(spectrogram_magnitudes_interp)

    # Plot 2: Spectrogram
    c = ax2.pcolormesh(f_common, frequencies, spectrogram_magnitudes,␣
↪shading='auto', cmap='magma')
```

```
    ax2.set_title("Full Sweep Spectrogram (All Frequencies in One Plot)")
    ax2.set_xlabel("Frequency (Hz)")
    ax2.set_ylabel("Target Sweep Frequency (Hz)")

    # Add a diagonal line to show perfect tracking
    ax2.plot(frequencies, frequencies, color='cyan', linestyle=':',␣
↪linewidth=2, label='Ideal Tracking')

    ax2.set_xlim(0, 1500)
    ax2.set_ylim(frequencies[0], frequencies[-1])
    ax2.grid(True, alpha=0.3)
    ax2.legend()

    fig.colorbar(c, ax=ax2, label='Magnitude (V)')

    plt.suptitle("Fourier Transform Analysis: Reconstruction Quality",␣
↪fontsize=14, fontweight='bold')
    plt.tight_layout(rect=[0, 0, 1, 0.96])
    plt.show()
```

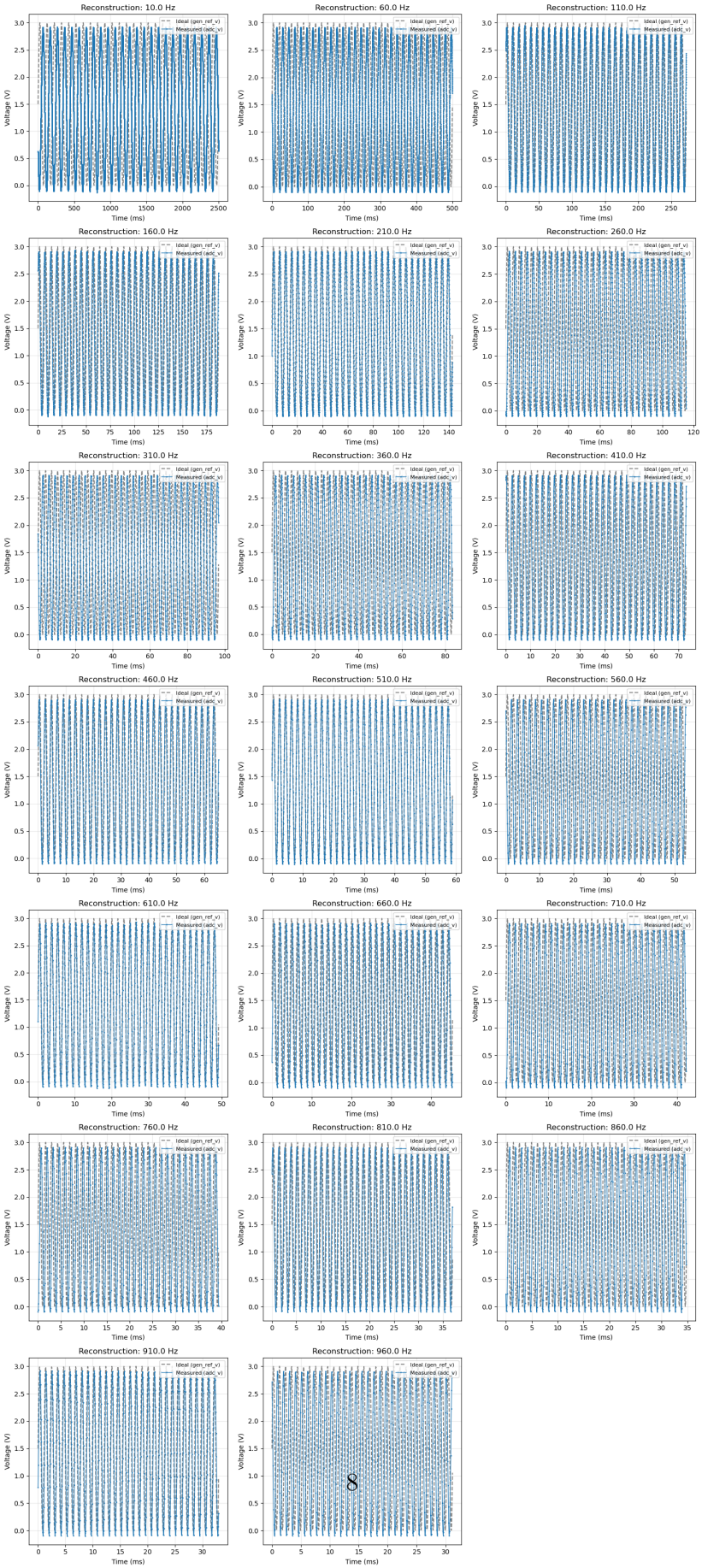
[4]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

def analyze_residuals_and_stats(df, target_freqs=None):
    """
    Calculates reconstruction statistics (RMSE, R-squared, SNR) and
    plots the residuals (ADC - Ideal) to visualize deviation.
    """
    frequencies = sorted(df['freq_hz'].unique())

    # If no specific frequencies are requested, pick low, mid, and high
    if target_freqs is None:
        target_freqs = [frequencies[0], frequencies[len(frequencies)//2],␣
↪frequencies[-1]]

    # Ensure requested frequencies actually exist in the data
    target_freqs = [f for f in target_freqs if f in frequencies]

    print(f"{'Freq (Hz)':<10} | {'RMSE (V)':<10} | {'Variance':<10} |␣
↪{'R-squared':<10} | {'SNR (dB)':<10}")
    print("-" * 65)

    stats_list = []

    # Create a grid for plotting: 2 rows (Signal + Residual) x N columns (for␣
↪each target freq)
```

```
    fig, axes = plt.subplots(2, len(target_freqs), figsize=(5 *␣
↪len(target_freqs), 6))
    if len(target_freqs) == 1:
        axes = np.array([[axes[0]], [axes[1]]]) # Force 2D array for consistent␣
↪indexing

    for col, freq in enumerate(target_freqs):
        segment = df[df['freq_hz'] == freq].copy()

        # Normalize time to start at 0 ms for this segment
        t_rel = segment['time_ms'] - segment['time_ms'].iloc[0]
        adc_v = segment['adc_v'].values
        ref_v = segment['gen_ref_v'].values

        # 1. Calculate Residuals
        residuals = adc_v - ref_v

        # 2. Calculate Statistics
        rmse = np.sqrt(np.mean(residuals**2))
        variance = np.var(residuals)

        # R-squared (Coefficient of Determination)
        ss_res = np.sum(residuals**2)
        ss_tot = np.sum((ref_v - np.mean(ref_v))**2)
        r_squared = 1 - (ss_res / ss_tot) if ss_tot != 0 else 0.0

        # Signal-to-Noise Ratio (SNR) in dB
        # Signal power = variance of ideal reference, Noise power = variance of␣
↪residuals
        signal_power = np.var(ref_v)
        noise_power = variance if variance > 0 else 1e-10
        snr_db = 10 * np.log10(signal_power / noise_power)

        stats_list.append({
            'Freq_Hz': freq, 'RMSE_V': rmse, 'Variance': variance,
            'R_Squared': r_squared, 'SNR_dB': snr_db
        })

        print(f"{freq:<10.1f} | {rmse:<10.4f} | {variance:<10.4f} | {r_squared:
↪<10.4f} | {snr_db:<10.2f}")

        # --- PLOTTING ---
        # Top row: Overlay of Ideal vs Measured
        ax_top = axes[0, col]
        ax_top.plot(t_rel, ref_v, label='Ideal (gen_ref_v)', color='gray',␣
↪linestyle='--', linewidth=1.5)
```

```
        ax_top.plot(t_rel, adc_v, label='Measured (adc_v)', color='tab:blue',␣
↪marker='.', markersize=2, linewidth=1)
        ax_top.set_title(f"Reconstruction: {freq:.1f} Hz")
        ax_top.set_ylabel("Voltage (V)")
        ax_top.legend(loc='upper right', fontsize=8)
        ax_top.grid(True, alpha=0.3)

        # Bottom row: Residuals (Deviation)
        ax_bot = axes[1, col]
        ax_bot.plot(t_rel, residuals, color='tab:red', linewidth=1, alpha=0.8)
        ax_bot.axhline(0, color='black', linestyle='-', linewidth=1)

        # Add +/- 1 Standard Deviation bands
        std_dev = np.std(residuals)
        ax_bot.axhline(std_dev, color='orange', linestyle=':', label=f'±1 ␣
↪({std_dev:.3f}V)')
        ax_bot.axhline(-std_dev, color='orange', linestyle=':')

        ax_bot.set_title(f"Residuals (ADC - Ideal)")
        ax_bot.set_xlabel("Time in segment (ms)")
        ax_bot.set_ylabel("Error (V)")
        ax_bot.legend(loc='upper right', fontsize=8)
        ax_bot.grid(True, alpha=0.3)

    plt.suptitle("Statistical Analysis: Reconstruction Quality & Residuals",␣
↪fontsize=14, fontweight='bold')
    plt.tight_layout(rect=[0, 0, 1, 0.96])
    plt.show()

    return pd.DataFrame(stats_list)
```

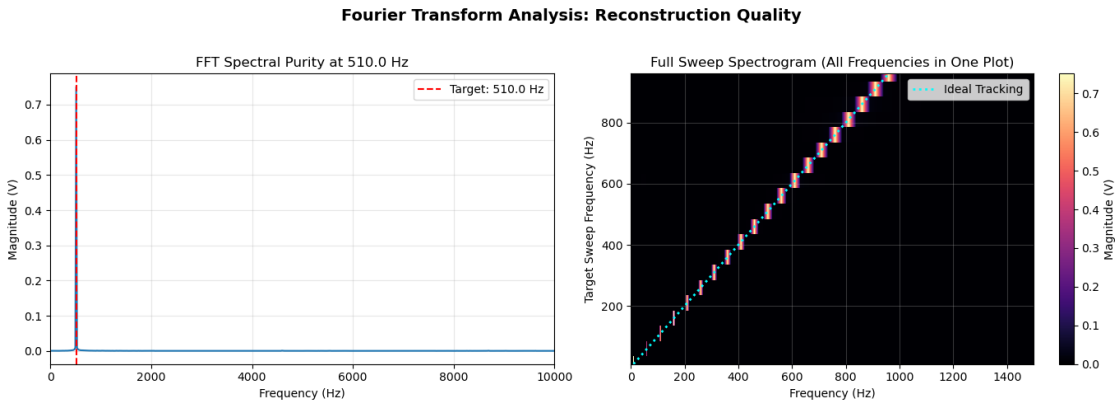
[5]:
```
plot_reconstructed_signals(dataframes['sweep_xiao_20261016_144657_50step.csv'],␣
↪20)
```

Signal Reconstruction: Measured ADC vs Ideal Generator Reference



[6]:
```
analyze_fft_and_spectrogram(dataframes['sweep_xiao_20261016_144657_50step.
↪csv'], target_freq_for_detail=510.0)
```

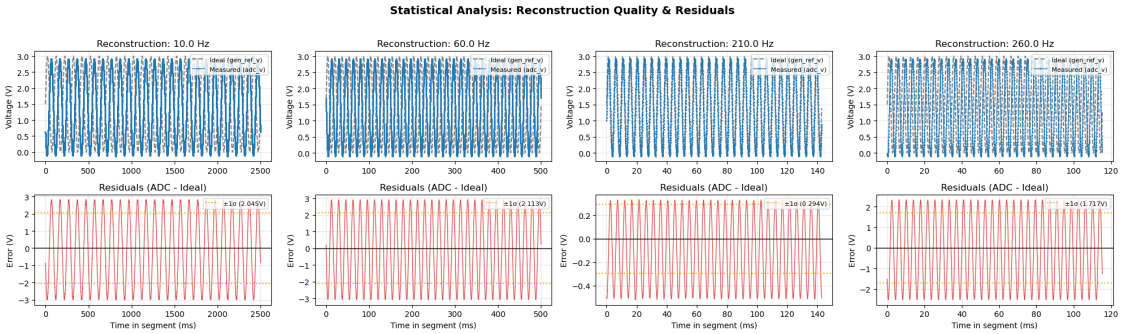
```
Target Freq (Hz) | Detected Peak (Hz) | Magnitude (V)    | Error (Hz)
-----------------------------------------------------------------
10.0             | 10.00              | 0.7511           | 0.00
60.0             | 60.00              | 0.7509           | 0.00
110.0            | 110.01             | 0.7510           | 0.01
160.0            | 160.00             | 0.7508           | 0.00
210.0            | 210.01             | 0.7510           | 0.01
260.0            | 260.08             | 0.7509           | 0.08
310.0            | 310.08             | 0.7510           | 0.08
360.0            | 360.14             | 0.7508           | 0.14
410.0            | 410.12             | 0.7509           | 0.12
460.0            | 460.12             | 0.7508           | 0.12
510.0            | 510.20             | 0.7507           | 0.20
560.0            | 560.22             | 0.7506           | 0.22
610.0            | 610.38             | 0.7505           | 0.38
660.0            | 660.07             | 0.7496           | 0.07
710.0            | 710.06             | 0.7511           | 0.06
760.0            | 760.46             | 0.7505           | 0.46
810.0            | 810.81             | 0.7502           | 0.81
860.0            | 860.83             | 0.7501           | 0.83
910.0            | 910.47             | 0.7507           | 0.47
960.0            | 960.00             | 0.7511           | 0.00
```



[7]:
```
# Example usage: Analyze low (10 Hz), mid (510 Hz), and high (960 Hz)␣
↪frequencies
# (Change these numbers to match frequencies that actually exist in your␣
↪specific CSV file)
```

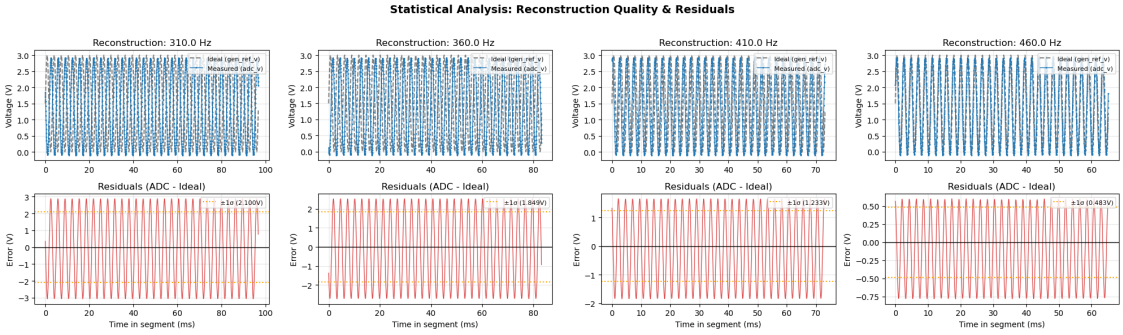
```
stats_df = analyze_residuals_and_stats(
    dataframes['sweep_xiao_20261016_144657_50step.csv'],
    target_freqs=[10.0, 60.0, 210.0, 260.0]
)
```

```
Freq (Hz)   | RMSE (V)    | Variance    | R-squared   | SNR (dB)
-----------------------------------------------------------------
10.0        | 2.0467      | 4.1805      | -2.7237     | -5.70
60.0        | 2.1153      | 4.4659      | -2.9775     | -5.99
210.0       | 0.3086      | 0.0865      | 0.9154      | 11.14
260.0       | 1.7196      | 2.9482      | -1.6275     | -4.18
```



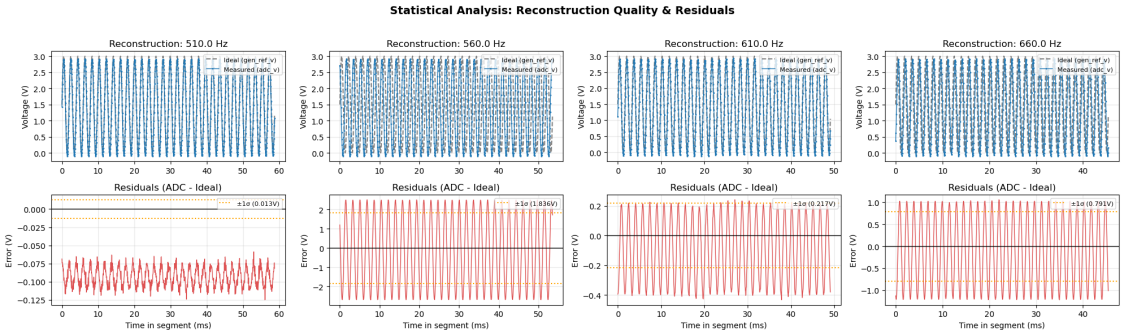
[8]:
```
stats_df = analyze_residuals_and_stats(
    dataframes['sweep_xiao_20261016_144657_50step.csv'],
    target_freqs=[310.0, 360.0, 410.0, 460.0]
)
```

```
Freq (Hz)   | RMSE (V)    | Variance    | R-squared   | SNR (dB)
-----------------------------------------------------------------
310.0       | 2.1016      | 4.4081      | -2.9252     | -5.93
360.0       | 1.8512      | 3.4183      | -2.0451     | -4.82
410.0       | 1.2368      | 1.5209      | -0.3594     | -1.31
460.0       | 0.4924      | 0.2336      | 0.7845      | 6.83
```



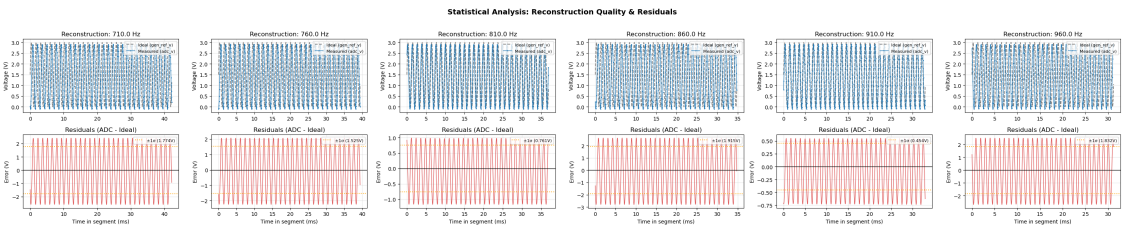
[9]:
```
stats_df = analyze_residuals_and_stats(
    dataframes['sweep_xiao_20261016_144657_50step.csv'],
    target_freqs=[510.0, 560.0, 610.0, 660.0]
)
```

```
Freq (Hz)   | RMSE (V)    | Variance    | R-squared   | SNR (dB)
-----------------------------------------------------------------
510.0       | 0.0945      | 0.0002      | 0.9921      | 38.27
560.0       | 1.8379      | 3.3691      | -2.0015     | -4.76
610.0       | 0.2366      | 0.0471      | 0.9503      | 13.79
660.0       | 0.7965      | 0.6258      | 0.4361      | 2.55
```

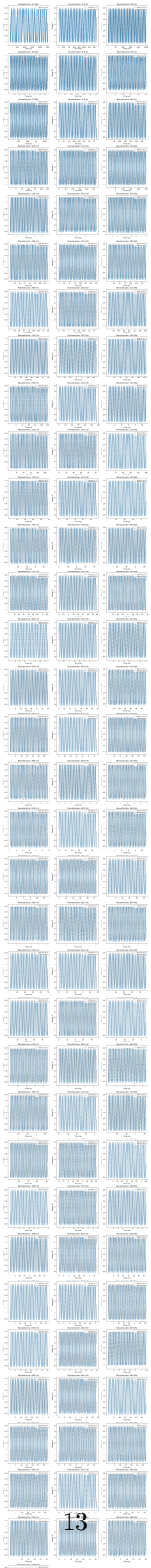


[10]:
```
stats_df = analyze_residuals_and_stats(
    dataframes['sweep_xiao_20261016_144657_50step.csv'],
    target_freqs=[ 710.0, 760.0, 810.0, 860.0, 910.0, 960.0]
)
```

```
Freq (Hz)   | RMSE (V)    | Variance    | R-squared   | SNR (dB)
-----------------------------------------------------------------
710.0       | 1.7761      | 3.1462      | -1.8040     | -4.47
760.0       | 1.5280      | 2.3264      | -1.0742     | -3.15
810.0       | 0.7671      | 0.5796      | 0.4774      | 2.88
860.0       | 1.9174      | 3.6679      | -2.2651     | -5.13
910.0       | 0.4632      | 0.2061      | 0.8093      | 7.37
960.0       | 1.8348      | 3.3577      | -1.9923     | -4.75
```



[11]:
```
plot_reconstructed_signals(dataframes['sweep_xiao_20261016_122345_10step.csv'],␣
↪100)
```



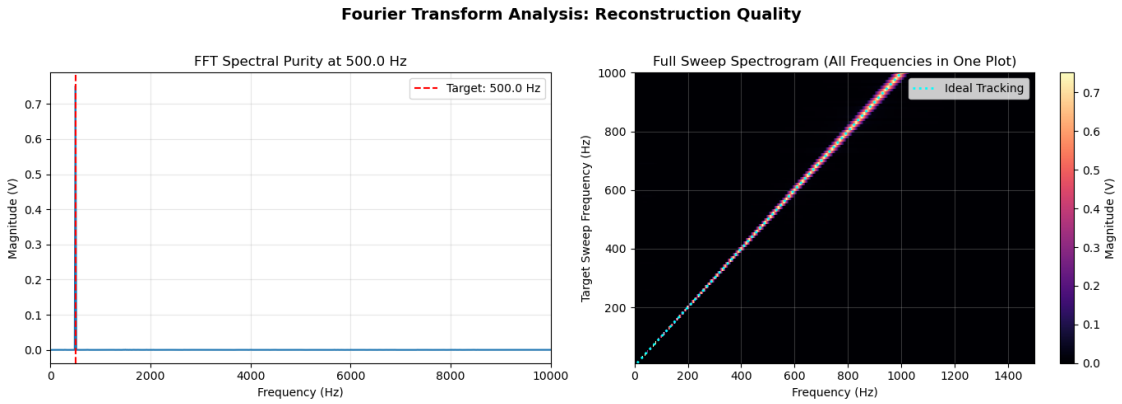
[12]:
```
analyze_fft_and_spectrogram(dataframes['sweep_xiao_20261016_122345_10step.
↪csv'], 500.0)
```

```
Target Freq (Hz) | Detected Peak (Hz) | Magnitude (V)   | Error (Hz)
-----------------------------------------------------------------
10.0             | 10.00              | 0.7512          | 0.00
20.0             | 20.00              | 0.7510          | 0.00
30.0             | 30.00              | 0.7506          | 0.00
40.0             | 40.00              | 0.7513          | 0.00
50.0             | 50.00              | 0.7508          | 0.00
60.0             | 60.00              | 0.7507          | 0.00
70.0             | 70.00              | 0.7515          | 0.00
80.0             | 80.00              | 0.7513          | 0.00
90.0             | 90.01              | 0.7510          | 0.01
100.0            | 100.00             | 0.7512          | 0.00
110.0            | 110.01             | 0.7513          | 0.01
120.0            | 120.00             | 0.7511          | 0.00
130.0            | 130.01             | 0.7512          | 0.01
140.0            | 140.02             | 0.7513          | 0.02
150.0            | 150.00             | 0.7511          | 0.00
160.0            | 160.00             | 0.7511          | 0.00
170.0            | 170.02             | 0.7510          | 0.02
180.0            | 180.02             | 0.7506          | 0.02
190.0            | 190.05             | 0.7512          | 0.05
200.0            | 200.00             | 0.7511          | 0.00
210.0            | 210.01             | 0.7512          | 0.01
220.0            | 220.02             | 0.7511          | 0.02
230.0            | 230.06             | 0.7509          | 0.06
240.0            | 240.00             | 0.7512          | 0.00
250.0            | 250.00             | 0.7514          | 0.00
260.0            | 260.08             | 0.7517          | 0.08
270.0            | 270.03             | 0.7512          | 0.03
280.0            | 280.11             | 0.7509          | 0.11
290.0            | 290.14             | 0.7509          | 0.14
300.0            | 300.00             | 0.7514          | 0.00
310.0            | 310.08             | 0.7510          | 0.08
320.0            | 320.00             | 0.7510          | 0.00
330.0            | 330.03             | 0.7512          | 0.03
340.0            | 340.14             | 0.7509          | 0.14
350.0            | 350.06             | 0.7513          | 0.06
360.0            | 360.14             | 0.7509          | 0.14
370.0            | 370.14             | 0.7509          | 0.14
380.0            | 380.23             | 0.7509          | 0.23
390.0            | 390.12             | 0.7510          | 0.12
400.0            | 400.00             | 0.7513          | 0.00
```

410.0	410.12	0.7510	0.12
420.0	420.17	0.7511	0.17
430.0	430.11	0.7511	0.11
440.0	440.21	0.7508	0.21
450.0	450.11	0.7509	0.11
460.0	460.12	0.7509	0.12
470.0	470.22	0.7510	0.22
480.0	480.00	0.7510	0.00
490.0	490.20	0.7501	0.20
500.0	500.00	0.7515	0.00
510.0	510.20	0.7495	0.20
520.0	520.38	0.7504	0.38
530.0	530.04	0.7510	0.04
540.0	540.05	0.7514	0.05
550.0	550.46	0.7507	0.46
560.0	560.22	0.7508	0.22
570.0	570.34	0.7508	0.34
580.0	580.27	0.7500	0.27
590.0	590.55	0.7501	0.55
600.0	600.00	0.7507	0.00
610.0	610.38	0.7511	0.38
620.0	620.48	0.7511	0.48
630.0	630.25	0.7513	0.25
640.0	640.34	0.7510	0.34
650.0	650.05	0.7512	0.05
660.0	660.07	0.7512	0.07
670.0	670.39	0.7507	0.39
680.0	680.27	0.7510	0.27
690.0	690.45	0.7511	0.45
700.0	700.12	0.7513	0.12
710.0	710.06	0.7514	0.06
720.0	720.29	0.7511	0.29
730.0	730.82	0.7501	0.82
740.0	740.74	0.7500	0.74
750.0	750.00	0.7513	0.00
760.0	760.46	0.7510	0.46
770.0	770.22	0.7511	0.22
780.0	780.23	0.7513	0.23
790.0	790.51	0.7508	0.51
800.0	800.00	0.7513	0.00
810.0	810.81	0.7497	0.81
820.0	820.79	0.7507	0.79
830.0	831.02	0.7500	1.02
840.0	840.34	0.7511	0.34
850.0	851.06	0.7498	1.06
860.0	860.83	0.7508	0.83
870.0	870.83	0.7503	0.83

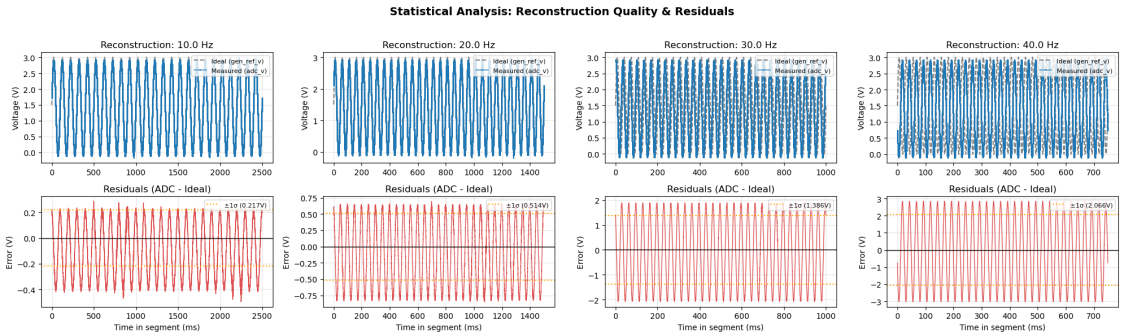
```
880.0           | 881.06            | 0.7500          | 1.06
890.0           | 890.21            | 0.7516          | 0.21
900.0           | 900.90            | 0.7505          | 0.90
910.0           | 910.47            | 0.7512          | 0.47
920.0           | 920.25            | 0.7514          | 0.25
930.0           | 930.23            | 0.7511          | 0.23
940.0           | 940.44            | 0.7513          | 0.44
950.0           | 950.87            | 0.7504          | 0.87
960.0           | 960.00            | 0.7508          | 0.00
970.0           | 970.87            | 0.7497          | 0.87
980.0           | 980.39            | 0.7511          | 0.39
990.0           | 990.10            | 0.7511          | 0.10
1000.0          | 1000.00           | 0.7515          | 0.00
```

```
/tmp/ipykernel_1543478/367295852.py:91: UserWarning: Creating legend with
loc="best" can be slow with large amounts of data.
  plt.tight_layout(rect=[0, 0, 1, 0.96])
```



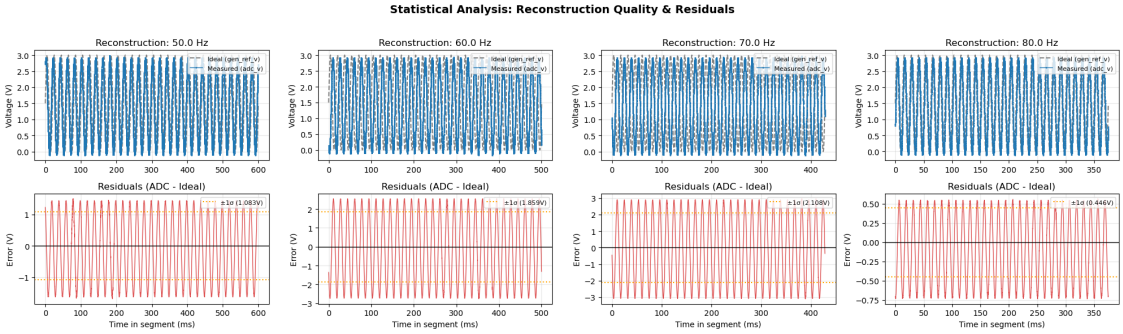
[13]:
```
stats_df = analyze_residuals_and_stats(
    dataframes['sweep_xiao_20261016_122345_10step.csv'],
    target_freqs=[10.0, 20.0, 30.0, 40.0]
)
```

```
Freq (Hz)   | RMSE (V)    | Variance    | R-squared   | SNR (dB)
------------------------------------------------------------------
10.0        | 0.2368      | 0.0472      | 0.9501      | 13.77
20.0        | 0.5228      | 0.2643      | 0.7570      | 6.29
30.0        | 1.3895      | 1.9218      | -0.7162     | -2.33
40.0        | 2.0680      | 4.2678      | -2.8013     | -5.79
```



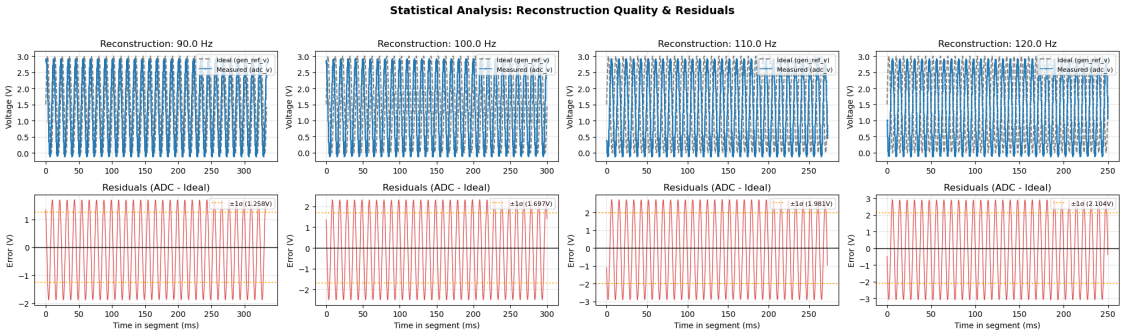
[14]:
```
stats_df = analyze_residuals_and_stats(
    dataframes['sweep_xiao_20261016_122345_10step.csv'],
    target_freqs=[50.0, 60.0, 70.0, 80.0]
)
```

Freq (Hz)	RMSE (V)	Variance	R-squared	SNR (dB)
50.0	1.0866	1.1719	-0.0496	-0.18
60.0	1.8618	3.4574	-2.0812	-4.88
70.0	2.1102	4.4440	-2.9579	-5.97
80.0	0.4556	0.1987	0.8155	7.53



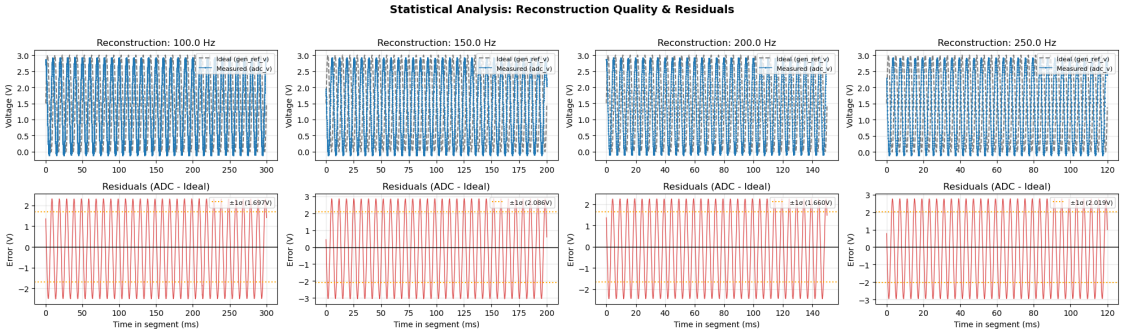
[15]:
```
stats_df = analyze_residuals_and_stats(
    dataframes['sweep_xiao_20261016_122345_10step.csv'],
    target_freqs=[90.0, 100.0, 110.0, 120.0]
)
```

Freq (Hz)	RMSE (V)	Variance	R-squared	SNR (dB)
90.0	1.2619	1.5835	-0.4152	-1.48
100.0	1.6993	2.8788	-1.5666	-4.08
110.0	1.9830	3.9234	-2.4949	-5.42
120.0	2.1064	4.4281	-2.9439	-5.95



[16]:
```
stats_df = analyze_residuals_and_stats(
    dataframes['sweep_xiao_20261016_122345_10step.csv'],
    target_freqs=[100.0, 150.0, 200.0, 250.0]
)
```

```
Freq (Hz)   | RMSE (V)    | Variance    | R-squared   | SNR (dB)
-----------------------------------------------------------------
100.0       | 1.6993      | 2.8788      | -1.5666     | -4.08
150.0       | 2.0878      | 4.3500      | -2.8746     | -5.87
200.0       | 1.6623      | 2.7544      | -1.4562     | -3.89
250.0       | 2.0215      | 4.0776      | -2.6323     | -5.59
```

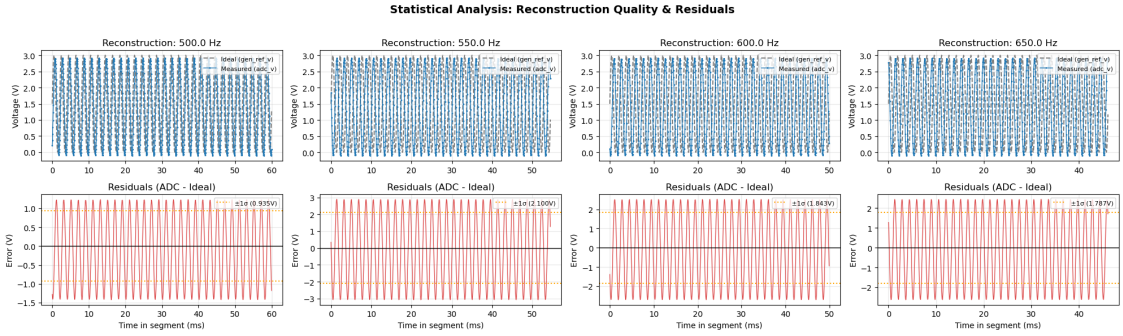


[17]:
```
stats_df = analyze_residuals_and_stats(
    dataframes['sweep_xiao_20261016_122345_10step.csv'],
    target_freqs=[300.0, 350.0, 400.0, 450.0]
)
```

```
Freq (Hz)   | RMSE (V)    | Variance    | R-squared   | SNR (dB)
-----------------------------------------------------------------
300.0       | 0.2001      | 0.0313      | 0.9644      | 15.56
350.0       | 1.1751      | 1.3718      | -0.2272     | -0.86
400.0       | 0.2136      | 0.0370      | 0.9594      | 14.83
450.0       | 1.5684      | 2.4511      | -1.1859     | -3.38
```

[18]:
```
stats_df = analyze_residuals_and_stats(
    dataframes['sweep_xiao_20261016_122345_10step.csv'],
    target_freqs=[500.0, 550.0, 600.0, 650.0]
)
```

```
Freq (Hz)   | RMSE (V)    | Variance    | R-squared   | SNR (dB)
-----------------------------------------------------------------
500.0       | 0.9393      | 0.8735      | 0.2158      | 1.10
550.0       | 2.1021      | 4.4100      | -2.9249     | -5.93
600.0       | 1.8452      | 3.3959      | -2.0263     | -4.80
650.0       | 1.7893      | 3.1926      | -1.8456     | -4.53
```

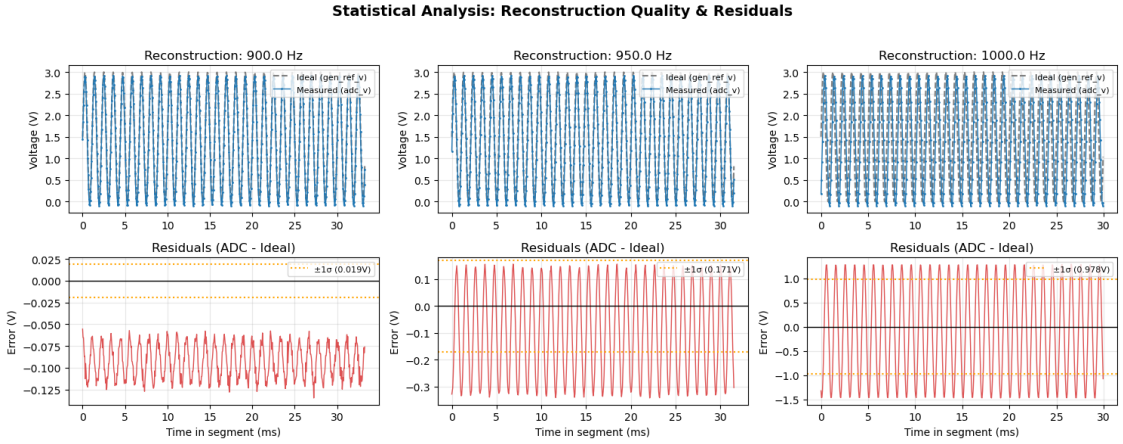


[19]:
```
stats_df = analyze_residuals_and_stats(
    dataframes['sweep_xiao_20261016_122345_10step.csv'],
    target_freqs=[700.0, 750.0, 800.0, 850.0]
)
```

```
Freq (Hz)   | RMSE (V)    | Variance    | R-squared   | SNR (dB)
-----------------------------------------------------------------
700.0       | 1.5766      | 2.4770      | -1.2093     | -3.43
750.0       | 0.1007      | 0.0013      | 0.9910      | 29.47
800.0       | 2.0700      | 4.2762      | -2.8088     | -5.80
850.0       | 0.1822      | 0.0246      | 0.9705      | 16.60
```

[20]:
```
stats_df = analyze_residuals_and_stats(
    dataframes['sweep_xiao_20261016_122345_10step.csv'],
    target_freqs=[900.0, 950.0, 1000.0]
)
```

```
Freq (Hz)   | RMSE (V)    | Variance    | R-squared   | SNR (dB)
-----------------------------------------------------------------
900.0       | 0.0947      | 0.0004      | 0.9920      | 34.85
950.0       | 0.1948      | 0.0292      | 0.9663      | 15.87
1000.0      | 0.9825      | 0.9567      | 0.1419      | 0.70
```



Shared helpers & constants

[21]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

LSB     = 3.6 / 4095.0          # 12-bit, 3.6 V full scale (SAADC gain 1/6)
Q_NOISE = LSB / np.sqrt(12)     # theoretical quantisation noise RMS  0.254 mV
```

```
def segments(df):
    """Yield (freq, t_seconds, adc, ref) per frequency segment."""
    for freq, seg in df.groupby('freq_hz', sort=True):
        t = (seg['time_ms'].values - seg['time_ms'].values[0]) / 1000.0
        yield freq, t, seg['adc_v'].values, seg['gen_ref_v'].values

def fit_sine(t, y, f):
    """Sub-sample-exact least-squares fit: y  C + amp*sin(2 ft + phase)."""
    w = 2*np.pi*f*t
    A = np.column_stack([np.ones_like(t), np.sin(w), np.cos(w)])
    C, M, N = np.linalg.lstsq(A, y, rcond=None)[0]
    return dict(offset=C, amp=np.hypot(M, N), phase=np.arctan2(N, M),
                fit=C + M*np.sin(w) + N*np.cos(w))
```

Phase-aligned residuals + quality statistics

[22]:
```
def phase_aligned_residual_stats(df, plot_freqs=None):
    rows, store = [], {}
    for freq, t, adc, ref in segments(df):
        fit = fit_sine(t, adc, freq)
        # Reference with COMMANDED amp/offset but FITTED phase -> phase error␣
 ↪removed
        ref_pa  = 1.5 + 1.5*np.sin(2*np.pi*freq*t + fit['phase'])
        res_pa  = adc - ref_pa        # noise + amplitude error
        res_fit = adc - fit['fit']    # noise + harmonics only (true noise␣
 ↪floor)
        rows.append(dict(freq_Hz=freq,
                         rmse_phase_aligned_V=np.sqrt(np.mean(res_pa**2)),
                         rmse_noise_floor_V  =np.sqrt(np.mean(res_fit**2)),
                         fitted_amp_V=fit['amp'],
                         fitted_phase_deg=np.degrees(fit['phase']) % 360,
                         R2=1 - np.sum(res_pa**2)/np.sum((ref_pa-ref_pa.
 ↪mean())**2),
                         SNR_dB=10*np.log10(np.var(ref_pa)/np.var(res_pa))))
        store[freq] = (t, res_pa, res_fit)

    stats = pd.DataFrame(rows)
    print(f"Theoretical quantisation noise RMS = {Q_NOISE*1000:.3f} mV")
    print(stats.to_string(index=False, float_format=lambda x: f"{x:.5f}"))

    pf = plot_freqs or [stats.freq_Hz.iloc[0], stats.freq_Hz.iloc[len(stats)//
 ↪2], stats.freq_Hz.iloc[-1]]
    pf = [f for f in pf if f in store]
    fig, axes = plt.subplots(2, len(pf), figsize=(5*len(pf), 6), squeeze=False)
    for col, f in enumerate(pf):
        t, rpa, rfit = store[f]
        axes[0, col].plot(t*1000, rpa*1000, color='tab:red', lw=0.8)
```

```
        axes[0, col].set_title(f"{f:.0f} Hz: residual vs phase-aligned ref")
        axes[0, col].set_ylabel("mV"); axes[0, col].grid(alpha=0.3)
        axes[1, col].plot(t*1000, rfit*1000, color='tab:green', lw=0.8)
        axes[1, col].axhline( Q_NOISE*1000, ls=':', c='k'); axes[1, col].
↪axhline(-Q_NOISE*1000, ls=':', c='k')
        axes[1, col].set_title(f"{f:.0f} Hz: noise floor (± quantisation =␣
↪dotted)")
        axes[1, col].set_xlabel("ms"); axes[1, col].set_ylabel("mV"); axes[1,␣
↪col].grid(alpha=0.3)
    plt.tight_layout(); plt.show()
    return stats
```

Aliasing demonstration by decimation

[23]:
```
def aliasing_demo(df, freq=1000.0, factors=(12, 20, 25)):
    seg = df[df['freq_hz'] == freq]
    t = (seg['time_ms'].values - seg['time_ms'].values[0]) / 1000.0
    y = seg['adc_v'].values
    fs = 1.0 / np.median(np.diff(t))

    fig, axes = plt.subplots(len(factors)+1, 2, figsize=(12,␣
↪3*(len(factors)+1)))
    for row, (yy, fsk, label) in enumerate(
            [(y, fs, f"original fs={fs:.0f} Hz")] +
            [(y[::k], fs/k, f"decimated x{k}: fs'={fs/k:.0f} Hz") for k in␣
↪factors]):
        N = len(yy)
        spec = np.abs(np.fft.rfft(yy - yy.mean()))/N
        fr   = np.fft.rfftfreq(N, 1/fsk)
        k_pk = np.argmax(spec[1:]) + 1
        n    = round(freq/fsk); f_alias = abs(freq - n*fsk)
        axes[row,0].plot(np.arange(N)/fsk*1000, yy, '.-', ms=2, lw=0.6)
        axes[row,0].set_title(f"{label}  (time domain)"); axes[row,0].
↪set_xlabel("ms"); axes[row,0].grid(alpha=0.3)
        axes[row,1].plot(fr, spec, lw=1)
        axes[row,1].axvline(f_alias, c='r', ls='--', label=f"predicted alias␣
↪{f_alias:.0f} Hz")
        axes[row,1].axvline(fr[k_pk], c='g', ls=':', label=f"measured peak␣
↪{fr[k_pk]:.0f} Hz")
        axes[row,1].set_title(f"{label}  (spectrum)"); axes[row,1].
↪set_xlabel("Hz"); axes[row,1].legend(fontsize=8); axes[row,1].grid(alpha=0.3)
        print(f"fs'={fsk:7.1f} Hz  Nyquist={fsk/2:6.1f} Hz  ->  {freq:.0f} Hz␣
↪folds to {f_alias:.1f} Hz "
              f"(measured peak {fr[k_pk]:.1f} Hz)")
    plt.tight_layout(); plt.show()
```

Sampling-interval proof (constant rate)

[24]:
```
def sampling_interval_check(df, fs_expected=20000):
    dts = np.concatenate([np.diff(t)*1000 for _, t, _, _ in segments(df)])  # ms
    print(f"intervals: mean={dts.mean():.6f} ms  std={dts.std():.6f} ms  "
          f"min={dts.min():.4f}  max={dts.max():.4f}  (expected {1000/
↪fs_expected:.4f} ms)")
    print(f"distinct interval values: {np.unique(dts.round(4)).size}")
    fig, ax = plt.subplots(1, 2, figsize=(11, 3.5))
    ax[0].hist(dts, bins=40, color='tab:blue'); ax[0].
↪set_title("Sampling-interval histogram (single bin = constant rate)")
    ax[0].set_xlabel("dt (ms)"); ax[0].grid(alpha=0.3)
    ax[1].plot(dts[::50], '.', ms=1); ax[1].set_title("dt vs sample index (no␣
↪drift/wander)")
    ax[1].set_xlabel("sample"); ax[1].set_ylabel("dt (ms)"); ax[1].grid(alpha=0.
↪3)
    plt.tight_layout(); plt.show()
```

Sweep-wide amplitude & frequency error

[25]:
```
def sweep_error_analysis(df, vpp=3.0):
    rows = []
    for freq, t, adc, ref in segments(df):
        N  = len(adc); fs = 1.0/np.median(np.diff(t))
        win = np.hanning(N); cg = win.mean()                    # coherent gain 0.5
        spec = np.abs(np.fft.rfft((adc-adc.mean())*win))/N/cg
        fr   = np.fft.rfftfreq(N, 1/fs)
        k = int(np.argmax(spec[1:])) + 1
        a, b, c = spec[k-1], spec[k], spec[min(k+1, len(spec)-1)]
        den = a - 2*b + c
        d = 0.5*(a-c)/den if den != 0 else 0.0                  # parabolic␣
↪sub-bin shift
        f_pk  = (k+d)*fs/N
        amp   = 2*(b - 0.25*(a-c)*d)                             # one-sided ->␣
↪amplitude
        rows.append(dict(freq_Hz=freq, N=N, bin_Hz=fs/N, f_peak_Hz=f_pk,
                         f_err_Hz=f_pk-freq, amp_V=amp,
                         amp_err_pct=100*(amp - vpp/2)/(vpp/2)))
    e = pd.DataFrame(rows)
    fig, ax = plt.subplots(2, 1, figsize=(9, 6), sharex=True)
    ax[0].errorbar(e.freq_Hz, e.f_err_Hz, yerr=e.bin_Hz/2, fmt='o-', ms=3, lw=0.
↪8, capsize=2)
    ax[0].axhline(0, c='k', lw=0.8); ax[0].set_ylabel("freq error (Hz)")
    ax[0].set_title("Peak-frequency error (bars = ±½ FFT bin)"); ax[0].
↪grid(alpha=0.3)
    ax[1].plot(e.freq_Hz, e.amp_err_pct, 'o-', ms=3, lw=0.8, color='tab:green')
    ax[1].axhline(0, c='k', lw=0.8); ax[1].set_ylabel("amplitude error (%)")
    ax[1].set_xlabel("commanded frequency (Hz)"); ax[1].set_title("Amplitude␣
↪flatness vs commanded 1.5 V"); ax[1].grid(alpha=0.3)
```

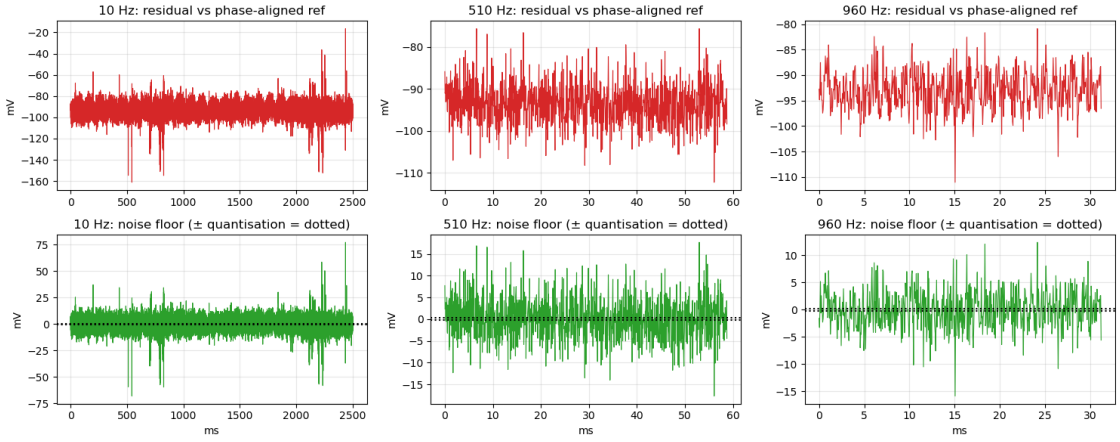
```
    plt.tight_layout(); plt.show()
    print(e.to_string(index=False, float_format=lambda x: f"{x:.4f}"))
    return e
```

[26]:
```
df = dataframes['sweep_xiao_20261016_144657_50step.csv']   # any of your files
```

[27]:
```
stats = phase_aligned_residual_stats(df)      # B: quality metrics + residual␣
↪plots
```

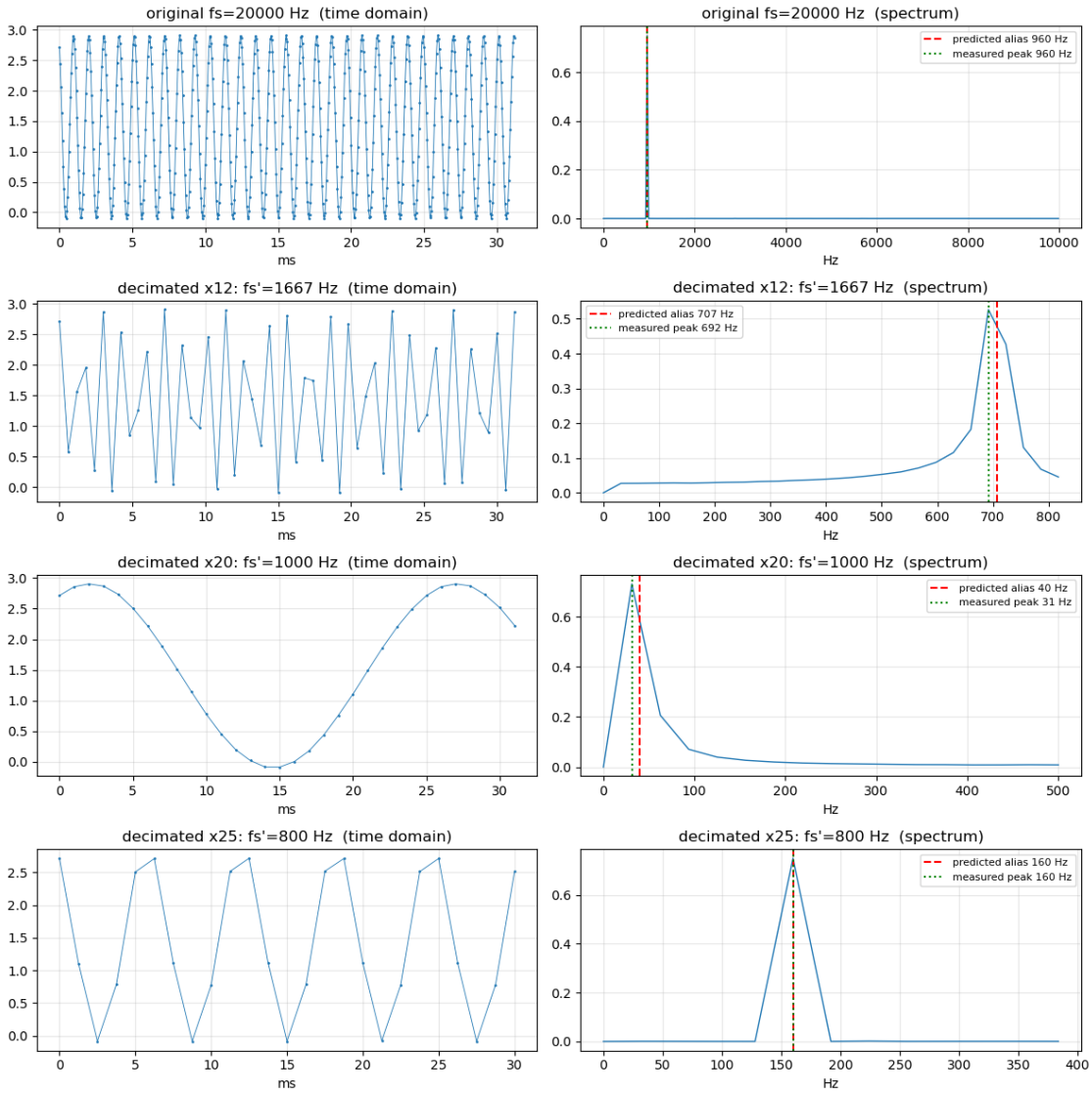
```
Theoretical quantisation noise RMS = 0.254 mV
  freq_Hz  rmse_phase_aligned_V  rmse_noise_floor_V  fitted_amp_V
fitted_phase_deg      R2   SNR_dB
 10.00000               0.09334             0.00544       1.50214
211.20385 0.99226 45.47409
 60.00000               0.09388             0.00445       1.50175
169.27151 0.99217 47.23009
110.00000               0.09388             0.00601       1.50204
46.40981 0.99217 44.68526
160.00000               0.09467             0.00688       1.50153
50.44818 0.99203 43.65677
210.00000               0.09350             0.00436       1.50190
344.07190 0.99223 47.32768
260.00000               0.09372             0.00457       1.50210
252.04373 0.99219 46.88008
310.00000               0.09376             0.00457       1.50191
162.98187 0.99219 46.94286
360.00000               0.09434             0.00478       1.50198
238.88176 0.99209 46.56370
410.00000               0.09370             0.00475       1.50215
71.03972 0.99219 46.56330
460.00000               0.09375             0.00469       1.50166
26.32994 0.99219 46.82638
510.00000               0.09370             0.00461       1.50146
0.65071 0.99220 47.02560
560.00000               0.09397             0.00438       1.50171
119.70298 0.99215 47.36473
610.00000               0.09540             0.01111       1.50146
348.28020 0.99191 39.56454
660.00000               0.09377             0.01027       1.49923
316.20053 0.99218 40.26478
710.00000               0.09267             0.00393       1.50224
246.66386 0.99237 47.96432
760.00000               0.09240             0.00394       1.50223
268.15654 0.99241 47.95165
810.00000               0.09291             0.00476       1.50220
42.05187 0.99233 46.51358
860.00000               0.09316             0.00379       1.50208
231.21039 0.99229 48.33535
```

```
910.00000                  0.09250              0.00398        1.50185
335.29659 0.99240 48.07866
960.00000                  0.09315              0.00355        1.50215
119.35036 0.99229 48.77680
```



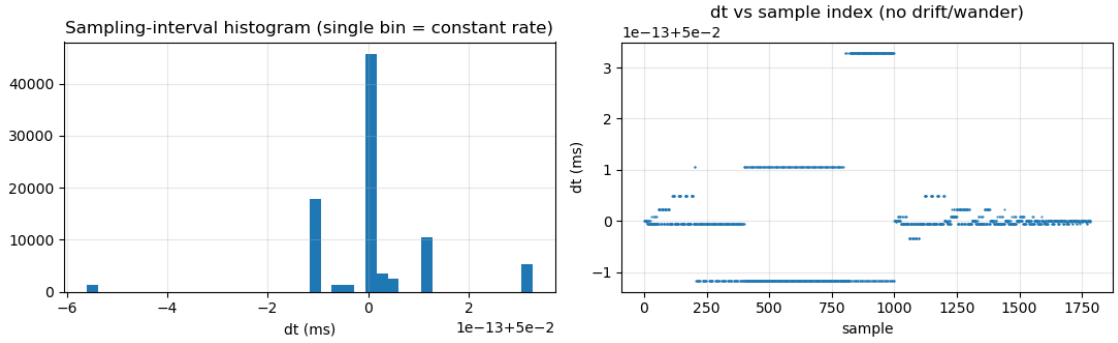
[31]:
```
aliasing_demo(df, freq=960.0)                    # C: Nyquist violation demo
```

```
fs'=20000.0 Hz  Nyquist=10000.0 Hz  ->  960 Hz folds to 960.0 Hz (measured peak
960.0 Hz)
fs'= 1666.7 Hz  Nyquist= 833.3 Hz  ->  960 Hz folds to 706.7 Hz (measured peak
691.8 Hz)
fs'= 1000.0 Hz  Nyquist= 500.0 Hz  ->  960 Hz folds to 40.0 Hz (measured peak
31.2 Hz)
fs'=  800.0 Hz  Nyquist= 400.0 Hz  ->  960 Hz folds to 160.0 Hz (measured peak
160.0 Hz)
```

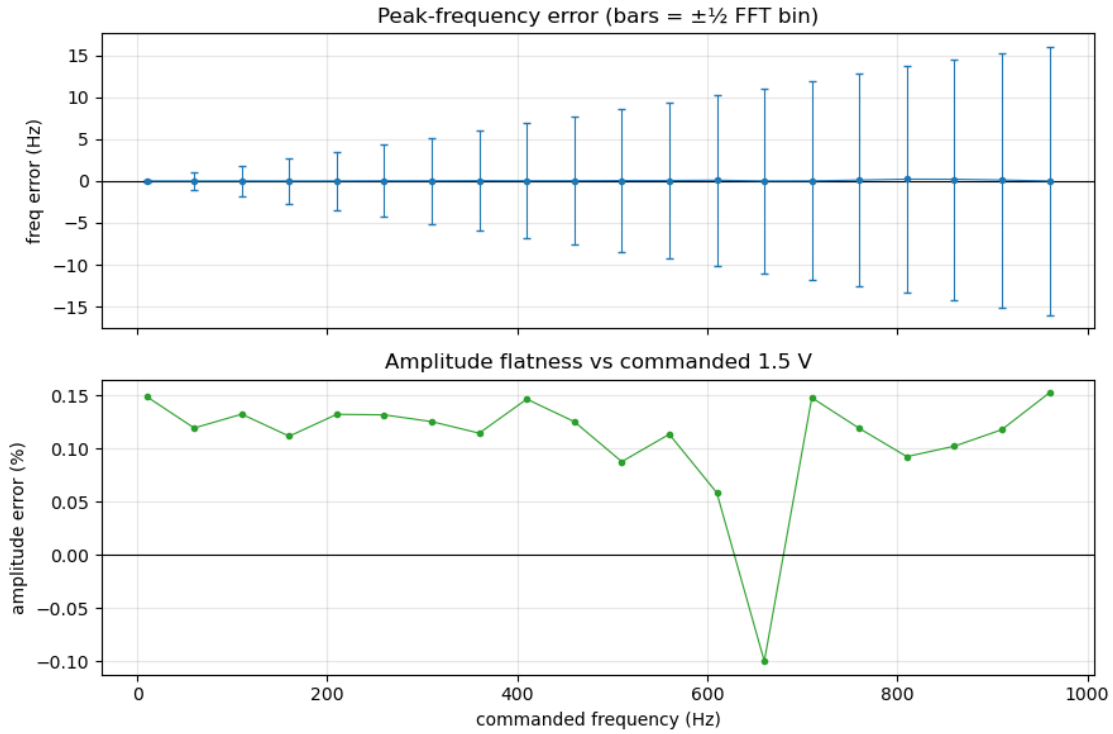


[32]:
```
sampling_interval_check(df)                    # D: constant-rate proof
```

```
intervals: mean=0.050000 ms  std=0.000000 ms  min=0.0500  max=0.0500  (expected
0.0500 ms)
distinct interval values: 1
```



[33]:
```
err    = sweep_error_analysis(df)
```



freq_Hz	N	bin_Hz	f_peak_Hz	f_err_Hz	amp_V	amp_err_pct
10.0000	50000	0.4000	9.9999	-0.0001	1.5022	0.1488
60.0000	10000	2.0000	59.9997	-0.0003	1.5018	0.1192
110.0000	5454	3.6670	110.0021	0.0021	1.5020	0.1322
160.0000	3750	5.3333	159.9991	-0.0009	1.5017	0.1115
210.0000	2857	7.0004	210.0020	0.0020	1.5020	0.1321
260.0000	2307	8.6693	260.0176	0.0176	1.5020	0.1316
310.0000	1935	10.3359	310.0190	0.0190	1.5019	0.1252
360.0000	1666	12.0048	360.0346	0.0346	1.5017	0.1144

```
410.0000  1463 13.6705   410.0244    0.0244 1.5022       0.1464
460.0000  1304 15.3374   460.0296    0.0296 1.5019       0.1252
510.0000  1176 17.0068   510.0503    0.0503 1.5013       0.0876
560.0000  1071 18.6741   560.0536    0.0536 1.5017       0.1134
610.0000   983 20.3459   610.0883    0.0883 1.5009       0.0584
660.0000   909 22.0022   660.0031    0.0031 1.4985      -0.1001
710.0000   845 23.6686   710.0119    0.0119 1.5022       0.1477
760.0000   789 25.3485   760.1070    0.1070 1.5018       0.1189
810.0000   740 27.0270   810.2147    0.2147 1.5014       0.0923
860.0000   697 28.6944   860.2034    0.2034 1.5015       0.1021
910.0000   659 30.3490   910.1116    0.1116 1.5018       0.1177
960.0000   625 32.0000   960.0008    0.0008 1.5023       0.1523
```

[34]:
```
df = dataframes['sweep_xiao_20261016_122345_10step.csv']   # any of your files
```

[35]:
```
stats = phase_aligned_residual_stats(df)      # B: quality metrics + residual␣
↪plots
```

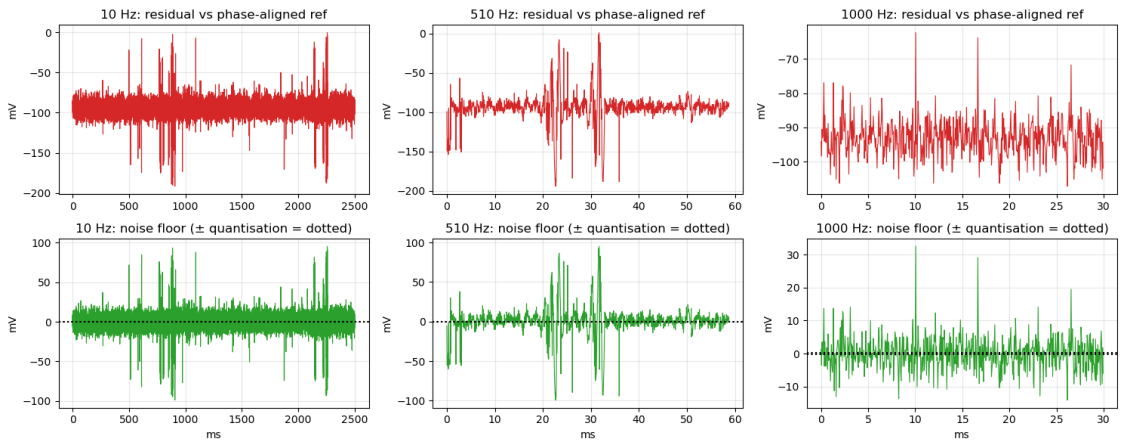
```
Theoretical quantisation noise RMS = 0.254 mV
   freq_Hz  rmse_phase_aligned_V  rmse_noise_floor_V  fitted_amp_V
fitted_phase_deg      R2   SNR_dB
  10.00000               0.09468             0.00936       1.50232
11.73879 0.99203 40.95673
  20.00000               0.09529             0.00905       1.50204
28.02816 0.99193 41.26954
  30.00000               0.09489             0.01011       1.50122
81.56914 0.99200 40.38746
  40.00000               0.09366             0.00818       1.50251
206.67224 0.99220 42.05265
  50.00000               0.09493             0.01142       1.50154
61.33051 0.99199 39.31936
  60.00000               0.09530             0.01243       1.50140
237.64792 0.99193 38.59546
  70.00000               0.09466             0.01046       1.50299
193.82262 0.99204 39.94485
  80.00000               0.09392             0.00595       1.50260
335.76053 0.99216 44.62734
  90.00000               0.09349             0.00500       1.50218
72.70928 0.99223 46.12973
 100.00000               0.09321             0.00541       1.50242
106.10456 0.99228 45.43932
 110.00000               0.09388             0.00551       1.50271
222.22997 0.99217 45.19720
 120.00000               0.09421             0.00568       1.50218
195.16805 0.99211 45.11647
 130.00000               0.09412             0.00671       1.50245
294.98520 0.99213 43.69714
 140.00000               0.09783             0.01767       1.50279
```

245.42004 0.99149 35.51305
 150.00000 0.09443 0.00606 1.50229
158.49714 0.99207 44.55933
 160.00000 0.09450 0.00516 1.50213
92.95742 0.99206 45.89718
 170.00000 0.09429 0.00610 1.50205
193.86894 0.99210 44.56404
 180.00000 0.09557 0.01422 1.50134
102.47665 0.99188 37.43144
 190.00000 0.09435 0.00538 1.50230
10.12454 0.99209 45.51763
 200.00000 0.09449 0.00519 1.50218
102.84820 0.99206 45.83351
 210.00000 0.09697 0.01550 1.50234
332.27690 0.99164 36.65487
 220.00000 0.09438 0.00525 1.50228
313.26724 0.99208 45.72523
 230.00000 0.09717 0.01553 1.50195
55.30557 0.99161 36.65111
 240.00000 0.09384 0.00493 1.50237
328.76935 0.99217 46.17604
 250.00000 0.09448 0.00910 1.50273
143.98908 0.99207 41.14129
 260.00000 0.09748 0.01933 1.50352
341.22913 0.99155 34.71538
 270.00000 0.09365 0.00534 1.50244
56.52042 0.99220 45.53622
 280.00000 0.09382 0.00521 1.50236
78.23185 0.99217 45.74160
 290.00000 0.09359 0.00524 1.50270
93.19140 0.99221 45.58564
 300.00000 0.09363 0.00512 1.50277
350.44646 0.99221 45.73678
 310.00000 0.09357 0.00512 1.50241
85.18125 0.99222 45.86398
 320.00000 0.09271 0.00514 1.50209
334.40174 0.99236 45.94029
 330.00000 0.09339 0.00500 1.50230
352.75946 0.99225 46.09420
 340.00000 0.09357 0.00490 1.50245
94.54306 0.99222 46.19604
 350.00000 0.09652 0.01986 1.50269
66.95086 0.99172 34.51057
 360.00000 0.09389 0.00745 1.50246
294.95420 0.99216 42.83643
 370.00000 0.09384 0.00482 1.50194
161.20126 0.99217 46.50994
 380.00000 0.09321 0.00490 1.50250

```
40.89658 0.99228 46.17412
 390.00000               0.09367             0.00481       1.50229
142.05783 0.99220 46.40483
 400.00000               0.09325             0.00504       1.50262
10.38666 0.99227 45.90995
 410.00000               0.09354             0.00537       1.50213
29.40853 0.99222 45.58134
 420.00000               0.09332             0.00512       1.50271
62.80452 0.99226 45.76115
 430.00000               0.09369             0.00551       1.50233
62.98680 0.99220 45.31863
 440.00000               0.09384             0.00544       1.50183
338.30192 0.99217 45.55824
 450.00000               0.09365             0.00564       1.50218
264.96890 0.99220 45.16930
 460.00000               0.09381             0.00592       1.50209
272.61046 0.99218 44.80254
 470.00000               0.09428             0.01106       1.50214
353.76195 0.99210 39.55538
 480.00000               0.09397             0.00491       1.50191
161.83587 0.99215 46.36740
 490.00000               0.09790             0.02216       1.50089
91.23784 0.99148 33.59386
 500.00000               0.09366             0.00485       1.50292
307.77513 0.99220 46.07281
 510.00000               0.09648             0.01998       1.49938
238.01695 0.99173 34.49607
 520.00000               0.09345             0.00499       1.50239
128.69247 0.99223 46.06974
 530.00000               0.09364             0.00502       1.50201
4.39666 0.99221 46.15464
 540.00000               0.09346             0.00516       1.50296
246.90232 0.99224 45.60232
 550.00000               0.09380             0.00506       1.50270
162.77798 0.99218 45.84648
 560.00000               0.09921             0.02297       1.50165
187.03208 0.99125 33.27949
 570.00000               0.09374             0.00610       1.50208
24.78531 0.99219 44.56194
 580.00000               0.09927             0.02436       1.50075
274.68768 0.99124 32.77547
 590.00000               0.09405             0.00488       1.50262
77.87664 0.99213 46.15196
 600.00000               0.09384             0.00529       1.50131
239.47301 0.99217 45.90819
 610.00000               0.09421             0.00489       1.50275
24.94434 0.99211 46.09463
 620.00000               0.09380             0.00484       1.50293
```

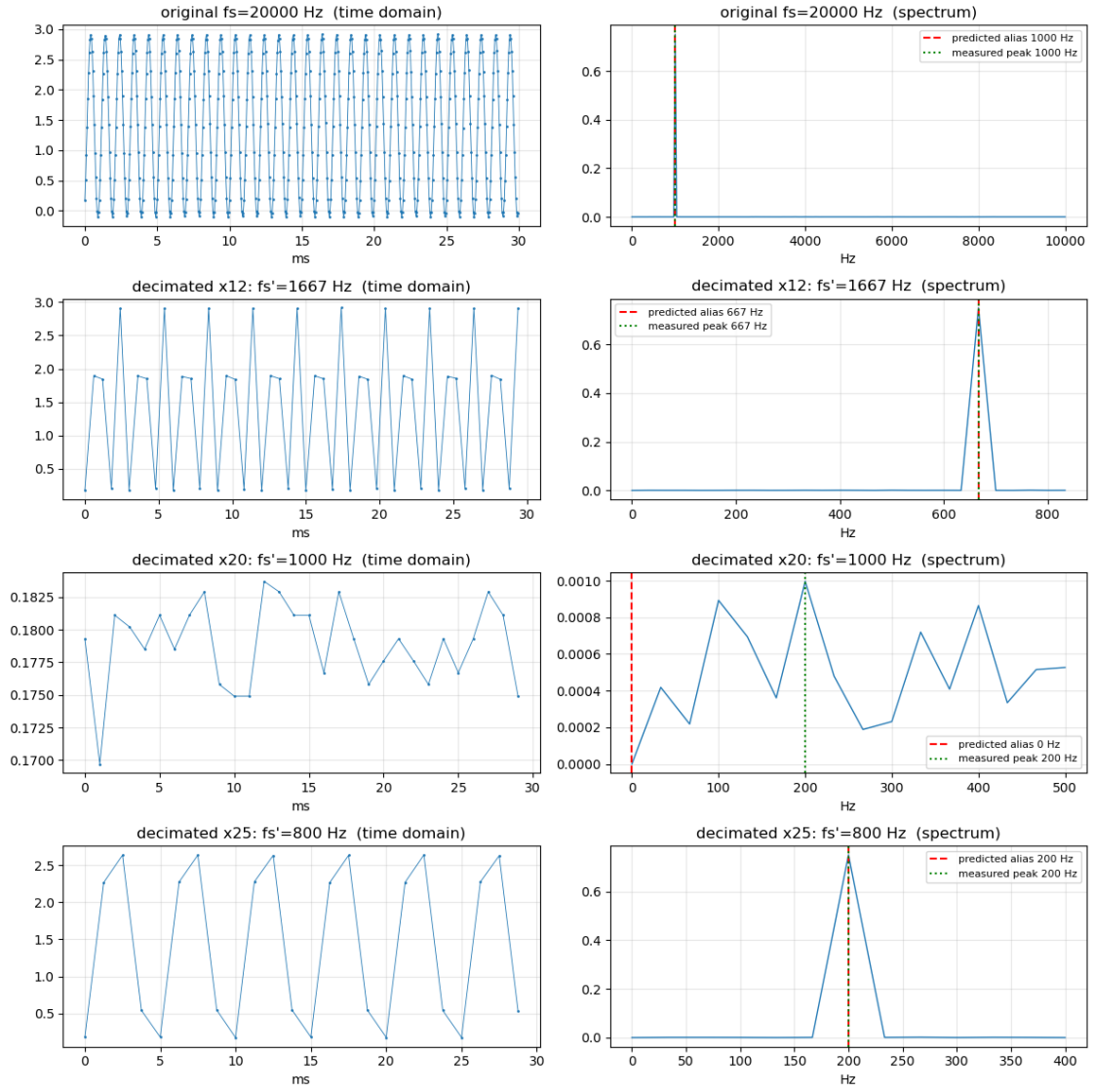
19.13905 0.99218 46.09158
 630.00000 0.09390 0.00548 1.50290
36.00965 0.99216 45.16869
 640.00000 0.09376 0.00496 1.50254
331.71137 0.99219 46.06874
 650.00000 0.09438 0.00509 1.50245
114.61926 0.99208 45.90195
 660.00000 0.09424 0.00495 1.50229
356.40911 0.99211 46.17168
 670.00000 0.09407 0.00573 1.50182
213.01470 0.99214 45.13852
 680.00000 0.09427 0.00491 1.50236
58.02965 0.99210 46.21333
 690.00000 0.09852 0.02707 1.50353
83.57662 0.99137 31.82357
 700.00000 0.09413 0.00501 1.50270
264.32714 0.99212 45.92249
 710.00000 0.09416 0.00493 1.50280
349.17361 0.99212 46.00349
 720.00000 0.09354 0.00468 1.50290
293.96959 0.99222 46.34955
 730.00000 0.09523 0.00703 1.50237
208.64115 0.99195 43.34162
 740.00000 0.09356 0.00463 1.50251
135.49344 0.99222 46.60933
 750.00000 0.09431 0.00493 1.50258
1.90365 0.99209 46.09762
 760.00000 0.09389 0.00477 1.50245
7.99779 0.99217 46.40916
 770.00000 0.09366 0.00502 1.50252
134.95248 0.99220 45.97419
 780.00000 0.09370 0.00502 1.50264
28.89375 0.99220 45.93120
 790.00000 0.09470 0.00650 1.50271
319.94873 0.99203 43.88740
 800.00000 0.09314 0.00487 1.50261
153.79629 0.99229 46.17819
 810.00000 0.09341 0.00506 1.50231
123.17290 0.99224 45.98883
 820.00000 0.09382 0.00505 1.50273
16.09754 0.99218 45.86210
 830.00000 0.09387 0.00472 1.50253
22.13138 0.99218 46.44795
 840.00000 0.09336 0.00511 1.50245
218.77383 0.99225 45.87464
 850.00000 0.09301 0.00512 1.50244
351.52032 0.99232 45.85776
 860.00000 0.09298 0.00455 1.50307

```
184.96118 0.99232 46.46013
 870.00000                0.09379                0.00721        1.50318
82.15476 0.99217 42.94819
 880.00000                0.09313                0.00527        1.50280
44.89292 0.99230 45.50496
 890.00000                0.09331                0.00631        1.50327
168.27611 0.99226 43.96883
 900.00000                0.09289                0.00510        1.50254
0.99455 0.99234 45.86390
 910.00000                0.09367                0.00817        1.50254
185.12408 0.99220 42.06260
 920.00000                0.09370                0.00486        1.50273
212.63978 0.99220 46.14671
 930.00000                0.09378                0.00496        1.50250
131.15811 0.99218 46.07524
 940.00000                0.09392                0.00543        1.50272
203.79130 0.99216 45.30178
 950.00000                0.09421                0.00551        1.50224
350.77377 0.99212 45.34907
 960.00000                0.09373                0.00494        1.50170
161.10019 0.99219 46.39472
 970.00000                0.09361                0.00503        1.50195
275.05165 0.99220 46.15815
 980.00000                0.09371                0.00484        1.50223
183.04372 0.99220 46.37831
 990.00000                0.09319                0.00488        1.50223
252.63895 0.99228 46.31324
1000.00000                0.09321                0.00501        1.50299
305.14652 0.99228 45.79846
```



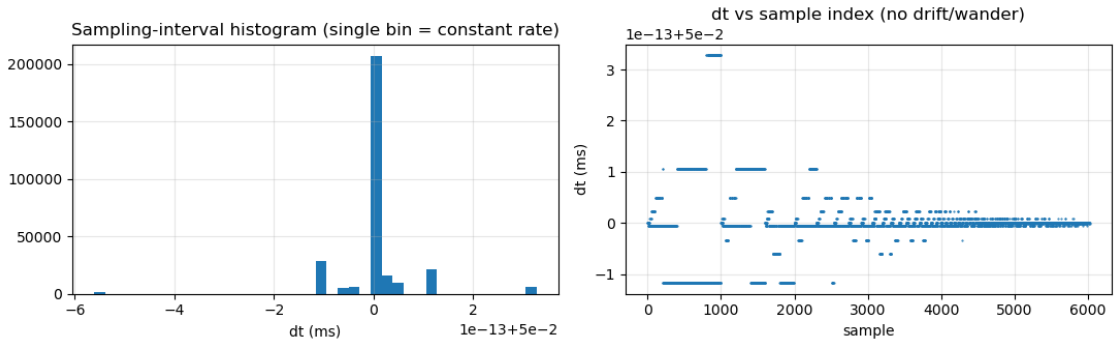
[36]:
```
aliasing_demo(df, freq=1000.0)                    # C: Nyquist violation demo
```

```
fs'=20000.0 Hz  Nyquist=10000.0 Hz  ->  1000 Hz folds to 1000.0 Hz (measured
peak 1000.0 Hz)
fs'= 1666.7 Hz  Nyquist= 833.3 Hz  ->  1000 Hz folds to 666.7 Hz (measured peak
666.7 Hz)
fs'= 1000.0 Hz  Nyquist= 500.0 Hz  ->  1000 Hz folds to 0.0 Hz (measured peak
200.0 Hz)
fs'=  800.0 Hz  Nyquist= 400.0 Hz  ->  1000 Hz folds to 200.0 Hz (measured peak
200.0 Hz)
```

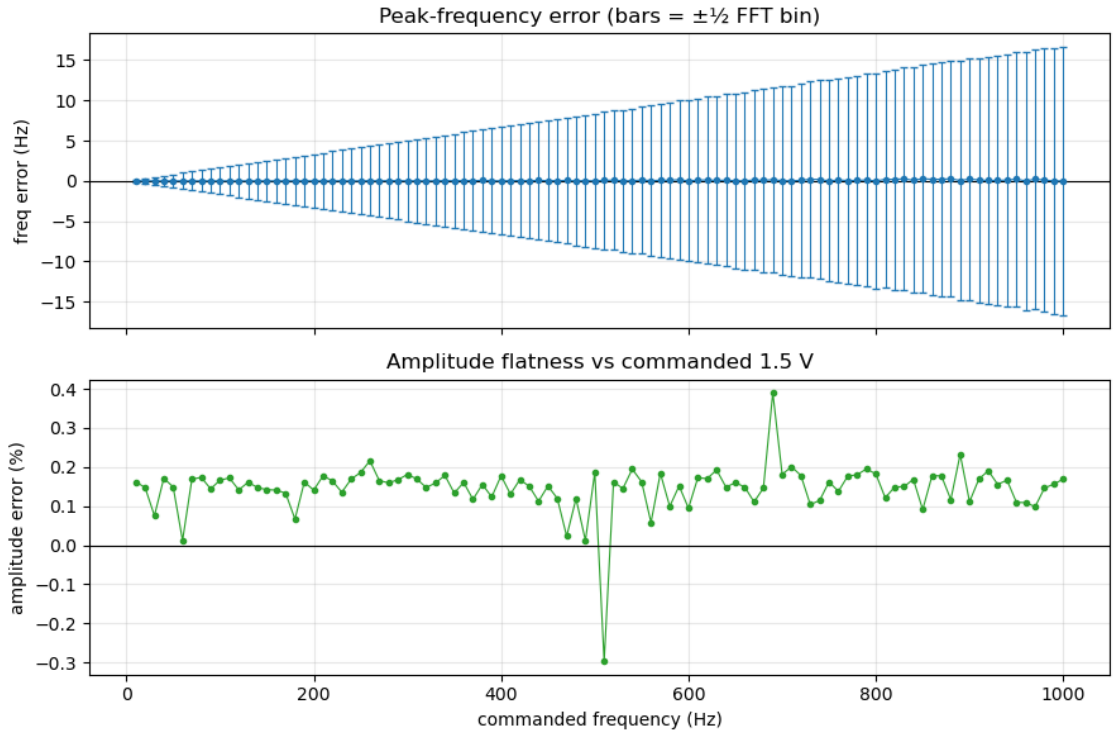


[37]:
```
sampling_interval_check(df)                    # D: constant-rate proof
```

```
intervals: mean=0.050000 ms  std=0.000000 ms  min=0.0500  max=0.0500  (expected
0.0500 ms)
distinct interval values: 1
```



[38]:
```
err   = sweep_error_analysis(df)
```



freq_Hz	N	bin_Hz	f_peak_Hz	f_err_Hz	amp_V	amp_err_pct
10.0000	50000	0.4000	9.9999	-0.0001	1.5024	0.1620
20.0000	30000	0.6667	19.9998	-0.0002	1.5022	0.1475
30.0000	20000	1.0000	29.9994	-0.0006	1.5011	0.0755
40.0000	15000	1.3333	39.9999	-0.0001	1.5026	0.1711
50.0000	12000	1.6667	50.0001	0.0001	1.5022	0.1488
60.0000	10000	2.0000	60.0004	0.0004	1.5002	0.0121
70.0000	8571	2.3335	70.0006	0.0006	1.5025	0.1688
80.0000	7500	2.6667	80.0001	0.0001	1.5026	0.1734

90.0000	6666	3.0003	90.0018	0.0018	1.5022	0.1441
100.0000	6000	3.3333	99.9999	-0.0001	1.5025	0.1664
110.0000	5454	3.6670	110.0018	0.0018	1.5026	0.1720
120.0000	5000	4.0000	119.9993	-0.0007	1.5021	0.1413
130.0000	4615	4.3337	130.0018	0.0018	1.5024	0.1610
140.0000	4285	4.6674	140.0061	0.0061	1.5022	0.1474
150.0000	4000	5.0000	149.9995	-0.0005	1.5021	0.1420
160.0000	3750	5.3333	159.9987	-0.0013	1.5021	0.1410
170.0000	3529	5.6673	170.0039	0.0039	1.5020	0.1329
180.0000	3333	6.0006	180.0023	0.0023	1.5010	0.0668
190.0000	3157	6.3351	190.0121	0.0121	1.5024	0.1615
200.0000	3000	6.6667	199.9990	-0.0010	1.5021	0.1408
210.0000	2857	7.0004	209.9997	-0.0003	1.5027	0.1776
220.0000	2727	7.3341	220.0042	0.0042	1.5025	0.1643
230.0000	2608	7.6687	230.0128	0.0128	1.5020	0.1354
240.0000	2500	8.0000	239.9987	-0.0013	1.5025	0.1692
250.0000	2400	8.3333	250.0012	0.0012	1.5028	0.1858
260.0000	2307	8.6693	260.0208	0.0208	1.5032	0.2160
270.0000	2222	9.0009	270.0048	0.0048	1.5025	0.1651
280.0000	2142	9.3371	280.0272	0.0272	1.5024	0.1594
290.0000	2068	9.6712	290.0332	0.0332	1.5025	0.1674
300.0000	2000	10.0000	299.9971	-0.0029	1.5027	0.1810
310.0000	1935	10.3359	310.0175	0.0175	1.5025	0.1694
320.0000	1875	10.6667	319.9987	-0.0013	1.5022	0.1476
330.0000	1818	11.0011	330.0061	0.0061	1.5024	0.1589
340.0000	1764	11.3379	340.0330	0.0330	1.5027	0.1785
350.0000	1714	11.6686	350.0122	0.0122	1.5020	0.1333
360.0000	1666	12.0048	360.0386	0.0386	1.5024	0.1604
370.0000	1621	12.3381	370.0321	0.0321	1.5018	0.1171
380.0000	1578	12.6743	380.0548	0.0548	1.5023	0.1555
390.0000	1538	13.0039	390.0264	0.0264	1.5018	0.1233
400.0000	1500	13.3333	399.9961	-0.0039	1.5027	0.1775
410.0000	1463	13.6705	410.0258	0.0258	1.5020	0.1306
420.0000	1428	14.0056	420.0374	0.0374	1.5025	0.1677
430.0000	1395	14.3369	430.0268	0.0268	1.5023	0.1508
440.0000	1363	14.6735	440.0517	0.0517	1.5017	0.1124
450.0000	1333	15.0038	450.0269	0.0269	1.5023	0.1514
460.0000	1304	15.3374	460.0291	0.0291	1.5018	0.1197
470.0000	1276	15.6740	470.0570	0.0570	1.5003	0.0230
480.0000	1250	16.0000	479.9950	-0.0050	1.5018	0.1189
490.0000	1224	16.3399	490.0353	0.0353	1.5002	0.0119
500.0000	1200	16.6667	499.9969	-0.0031	1.5028	0.1862
510.0000	1176	17.0068	510.0587	0.0587	1.4955	-0.2967
520.0000	1153	17.3461	520.0918	0.0918	1.5024	0.1620
530.0000	1132	17.6678	530.0103	0.0103	1.5022	0.1448
540.0000	1111	18.0018	540.0131	0.0131	1.5029	0.1946
550.0000	1090	18.3486	550.1125	0.1125	1.5024	0.1620
560.0000	1071	18.6741	560.0339	0.0339	1.5009	0.0581

570.0000	1052	19.0114	570.0734	0.0734	1.5028	0.1838
580.0000	1034	19.3424	580.0777	0.0777	1.5015	0.0992
590.0000	1016	19.6850	590.1378	0.1378	1.5023	0.1515
600.0000	1000	20.0000	600.0026	0.0026	1.5014	0.0947
610.0000	983	20.3459	610.0913	0.0913	1.5026	0.1724
620.0000	967	20.6825	620.1139	0.1139	1.5026	0.1703
630.0000	952	21.0084	630.0568	0.0568	1.5029	0.1925
640.0000	937	21.3447	640.0831	0.0831	1.5022	0.1486
650.0000	923	21.6685	650.0117	0.0117	1.5024	0.1598
660.0000	909	22.0022	660.0101	0.0101	1.5022	0.1490
670.0000	895	22.3464	670.1073	0.1073	1.5017	0.1106
680.0000	882	22.6757	680.0610	0.0610	1.5022	0.1463
690.0000	869	23.0150	690.1468	0.1468	1.5058	0.3898
700.0000	857	23.3372	700.0243	0.0243	1.5027	0.1812
710.0000	845	23.6686	710.0108	0.0108	1.5030	0.2007
720.0000	833	24.0096	720.0639	0.0639	1.5027	0.1783
730.0000	821	24.3605	730.1942	0.1942	1.5016	0.1039
740.0000	810	24.6914	740.1750	0.1750	1.5017	0.1145
750.0000	800	25.0000	750.0008	0.0008	1.5024	0.1619
760.0000	789	25.3485	760.1116	0.1116	1.5021	0.1376
770.0000	779	25.6739	770.0446	0.0446	1.5027	0.1767
780.0000	769	26.0078	780.0561	0.0561	1.5027	0.1803
790.0000	759	26.3505	790.1159	0.1159	1.5029	0.1950
800.0000	750	26.6667	799.9943	-0.0057	1.5027	0.1822
810.0000	740	27.0270	810.1932	0.1932	1.5018	0.1212
820.0000	731	27.3598	820.1879	0.1879	1.5022	0.1487
830.0000	722	27.7008	830.2520	0.2520	1.5022	0.1497
840.0000	714	28.0112	840.0844	0.0844	1.5025	0.1684
850.0000	705	28.3688	850.2626	0.2626	1.5014	0.0912
860.0000	697	28.6944	860.2051	0.2051	1.5027	0.1780
870.0000	689	29.0276	870.1931	0.1931	1.5027	0.1771
880.0000	681	29.3686	880.2637	0.2637	1.5018	0.1167
890.0000	674	29.6736	890.0495	0.0495	1.5035	0.2319
900.0000	666	30.0300	900.2161	0.2161	1.5017	0.1113
910.0000	659	30.3490	910.1139	0.1139	1.5025	0.1692
920.0000	652	30.6748	920.0512	0.0512	1.5029	0.1906
930.0000	645	31.0078	930.0573	0.0573	1.5023	0.1556
940.0000	638	31.3480	940.1015	0.1015	1.5025	0.1663
950.0000	631	31.6957	950.2134	0.2134	1.5016	0.1097
960.0000	625	32.0000	959.9931	-0.0069	1.5016	0.1087
970.0000	618	32.3625	970.2155	0.2155	1.5015	0.0983
980.0000	612	32.6797	980.0955	0.0955	1.5022	0.1462
990.0000	606	33.0033	990.0044	0.0044	1.5023	0.1563
1000.0000	600	33.3333	999.9896	-0.0104	1.5025	0.1689